

■ VIDEO DIALOGUE LOCALIZATION SYSTEM

Combined Technical Documentation, System Design & V2 Optimization Benchmarks

Document Version:	2.0 (Production)
Pipeline Engine:	V2 Coarse-to-Fine ASR (base/tiny -> small)
Web UI Framework:	Streamlit Web Dashboard
Output Document:	Video_Dialogue_Localization_Documentation.pdf

Included Documentation Sections:

1. Project Overview & Quickstart	README.md
2. V2 Optimization & Benchmarks	docs\optimization_and_benchmarks_v2.md
3. V1 System Documentation	docs\system_documentation_v1.md
4. Core Architecture Design	design.md
5. Dialogue Matching Design	matching_design.md
6. Frame Extraction Design	frame_extraction_design.md

SECTION: 1. PROJECT OVERVIEW & QUICKSTART

Video Dialogue Localization System — System Technical Manual

An end-to-end Python pipeline for ingesting public video URLs (YouTube, OK.ru, Vimeo, direct streams), extracting 16kHz mono PCM WAV speech audio, and generating segment & word-level timestamped transcripts using Faster-Whisper.

[!NOTE]

■ **Full System Documentation:** See `system_documentation.md` for complete architecture, benchmarking, and component details.

■ System Overview

The system processes public video content across structured pipeline phases to localize target dialogue queries to exact video frames:



[illegible]

■ ■ Module Architecture

1. Ingestion Module (src/ingestion/)

- **ingest_video(url, output_dir=None, proxy=None)**: Public entry point.
- **downloader.py**: Invokes **yt-dlp** as a subprocess with format **bestvideo[height<=720]+bestaudio/best**. Includes direct media stream fallback (**ffmpeg -c copy**).
- **probe.py**: Runs **ffprobe** to validate stream counts, duration > 0, captures rational frame rates ("24000/1001"), and detects VFR/CFR (**is_vfr**).

2. Audio Module (src/audio/)

- `extract_audio(video_path, output_path=None)`: Public entry point.
- `extractor.py`: Executes `ffmpeg -y -i <video> -vn -ac 1 -ar 16000 -c:a pcm_s16le <wav>` to convert audio into 16kHz 1-channel mono PCM WAV.
- `probe.py`: Runs `ffprobe` to validate `sample_rate == 16000`, `channels == 1`, `duration > 0`, and `codec_name == "pcm_s16le"`.

3. ASR Speech Recognition Module (src/asr/)

- `transcribe_audio(audio_path, model_size="base", device="cpu", compute_type="int8", language=None)`: Public API.
- `transcriber.py`: Loads `WhisperModel` from `faster-whisper` and executes CTranslate2-accelerated inference on CPU with `word_timestamps=True` and VAD filtering.
- `models.py`: Dataclasses `WordTimestamp`, `TranscriptSegment`, `TranscriptionResult`.
- `errors.py`: Exception hierarchy (`ASRError`, `AudioNotFoundError`, `ModelLoadError`, `TranscriptionError`).

■ Python API Usage Guide

1. Ingesting Video (Phase 2)

```
from ingestion import ingest_video

result = ingest_video("https://ok.ru/video/248244667877", output_dir="output")
print(f"Video saved to: {result.video_path}")
print(f"Duration: {result.metadata.duration}s")
```

2. Extracting Audio (Phase 3)

```
from audio import extract_audio
```

```
result = extract_audio("output/248244667877.mp4")
print(f"Audio saved to: {result.audio_path}")
print(f"Sample Rate: {result.metadata.sample_rate} Hz")
```

3. Transcribing Audio to Timestamped Text (Phase 4 ASR)

```
from asr import transcribe_audio

# Transcribe 16kHz WAV audio using Faster-Whisper on CPU
result = transcribe_audio(
    audio_path="output/248244667877.wav",
    model_size="base", # 'tiny', 'base', 'small', 'medium', 'large-v3'
    device="cpu", # CPU execution
    compute_type="int8" # int8 precision for fast CPU inference
)

print(f"Detected Language: {result.language} (prob={result.language_probability:.2f})")
print(f"Total Segments: {len(result.segments)}")
print(f"Full Text: {result.full_text[:100]}...")

### 4. Matching Dialogue to Timestamps (Phase 5)
```

```
from matching import match_dialogue
```

Match target dialogue query against transcript JSON or object

```
result = match_dialogue(
    target_text="My mind rebels at stagnation",
    transcript="output/248244667877_transcript_small.json",
    confidence_threshold=80.0
)
```

5. Extracting Video Frame at Dialogue Timestamp (Phase 6)

```
from frame_extraction import extract_frame

# Extract video frame at matched dialogue timestamp (e.g. 320.48s)
frame_res = extract_frame(
    video_path="output/248244667877.mp4",
    timestamp=result.start_time, # 320.48s
    output_dir="output/frames"
)

print(f"Extracted Frame: {frame_res.frame_path}")
print(f"Resolution: {frame_res.width}x{frame_res.height}")
print(f"Extraction Time: {frame_res.extraction_duration_seconds}s")
```

■ Verification Benchmarks (Supplied OK.ru Video)

Target URL

<https://ok.ru/video/248244667877> (*The Adventures of Sherlock Holmes: A Scandal in Bohemia*)

Phase 4 ASR Speech Recognition Results

- **Status:** PASSED ■
 - **Detected Language:** **en** (English, probability = 0.96)
 - **Audio Duration Transcribed:** **3,261.78 seconds** (~54 minutes)
 - **Total Segments Transcribed:** **623 segments**
 - **Word Timestamps:** **Extracted for every word**
 - **Model Used:** **tiny / base** (Faster-Whisper on CPU with **int8**)
 - **Transcription Time:** **123.1 seconds** (2 min 3 sec)
-

■ Test Suite Guide

Run Unit Tests (45/45 PASSED in 0.15s)

```
.\.venv\Scripts\python.exe -m pytest tests/test_unit.py tests/test_audio_unit.py tests/test_asr_unit.py -v
```

Run Integration Tests (Real Media & Faster-Whisper Model)

```
.\.venv\Scripts\python.exe -m pytest tests/test_integration.py tests/test_audio_integration.py tests/test_asr_integration.py -v -m integration
```

SECTION: 2. V2 OPTIMIZATION & BENCHMARKS

Video Dialogue Localization System — V2 Technical Documentation & Optimization Benchmarks

Executive Summary

This document provides a comprehensive technical reference for the **Video Dialogue Localization System (V2 Optimization)**. It documents the core architecture, optimization phases, empirical benchmarking results, caching strategies, and CLI interfaces implemented to achieve **62% faster cold-run processing** and **sub-second (<0.25s) warm query response times** while maintaining **100.0% dialogue localization accuracy**.

System Architecture

```
graph TD
    subgraph Mode_Selection [Mode Selection]
        A[User Input: Video URL / Path + Query] --> B{ASR Mode}
    end

    subgraph Standard_V1_Pipeline [Standard V1 Pipeline mode='standard']
        B -->|standard| C1[Full Audio Transcription: Faster-Whisper small + word_timestamps=True]
        C1 --> C2[RapidFuzz Sliding Window Matching]
        C2 --> C3[20ms Pre-Roll Frame Extraction]
    end

    subgraph Coarse_to_Fine_V2_Pipeline [Coarse-to-Fine V2 Pipeline mode='coarse_to_fine']
        B -->|coarse_to_fine| D1{Stage 1: Coarse Cache Exists?}
        D1 -->|No| D2[Stage 1: Coarse ASR - Faster-Whisper base + word_timestamps=False]
        D1 -->|Yes| D3[Load _transcript_coarse_base.json - 0.01s]
        D2 --> D4[Save _transcript_coarse_base.json]
        D4 --> D5[Stage 2: Top-K Candidate Region Search top_k=3 + 5s padding]
        D3 --> D5
        D5 --> D6{Stage 3: Fine Cache Exists?}
        D6 -->|Yes| D7[Load _v2_fine_cache.json - 0.01s]
        D6 -->|No| D8[FFmpeg Slice Candidate WAV + Fine ASR - Faster-Whisper small + word_timestamps=True]
        D7 --> D9[Save _v2_fine_cache.json]
        D8 --> D9
        D9 --> E[Dialogue Matching & Frame Extraction]
        E --> F
    end

    C3 --> F[LocalizationResult: HMS Timestamp, Frame Image, Confidence]
    E --> F
```

Key Optimization Breakthroughs & Empirical Benchmarks

Optimization 1: Multi-Core Parallel Audio Chunk ASR (Phase 7)

- **Problem:** Sequential transcription of long audio files (>30 minutes) on CPU was constrained by single-threaded CTranslate2 decoding bottlenecks.

- **Implementation:**

Audio file is sliced into 10-minute 16kHz mono WAV chunks using sample-accurate `-c:a pcm_s16le` codec.

Multi-core execution uses a single shared `WhisperModel(num_workers=2)` instance bound to `ThreadPoolExecutor`.

OpenMP/MKL thread oversubscription memory spikes resolved by binding `OMP_NUM_THREADS = 10` and `MKL_NUM_THREADS = 10`.

- **Empirical Benchmark (33.6-Minute YouTube Video [Y3_jS-q0Lkw](#)):**

Sequential Processing (Before): 624.55 seconds (10.4 minutes)

Multi-Core Parallel Processing (After): 563.06 seconds (9.3 minutes)

Time Saved: 61.49 seconds faster

RAM Footprint: ~461 MB (*Shared weight matrix*)

Optimization 2: Coarse-to-Fine Two-Stage ASR Pipeline (V2 Extension)

- **Problem:** Running full word-timestamped ASR at high resolution (`small` model) across an entire 54-minute video when the query quote is only 3 seconds long wastes CPU cycles.

- **Implementation:**

Stage 1 (Coarse Search): Fast segment-level transcription using `base` model (`word_timestamps=False`, `vad_filter=True`).

Stage 2 (Top-K Candidate Search): RapidFuzz sequence search against coarse segments to find top- K candidate regions ($K=3$) with a 5.0s safety padding buffer.

Stage 3 (Fine Sub-Region ASR): FFmpeg extracts only the 17-second candidate WAV sub-region and transcribes it with the `small` model (`word_timestamps=True`). Timestamps are re-offset (`word.start + pad_start`).

- **Empirical Benchmark (54-Minute Sherlock Holmes Video [248244667877.mp4](#)):**

V1 Standard Full Audio ASR: 419.18 seconds (7.0 minutes)

V2 Coarse-to-Fine Cold Run: 159.08 seconds (2.6 minutes)

Speedup: 62.0% Cold-Run Execution Speedup (Saved 4.3 minutes!)

Accuracy: 100.0% Match Confidence (Timestamp delta 0.14s, 4 frames difference).

Optimization 3: Dual-Layer Disk Caching & Memory Lifecycle Management

- **Problem:** Live ASR inference on repeat or warm CLI queries caused 160s+ execution times.

- **Implementation:**

Layer 1 (Coarse Cache): `output/<video_stem>_transcript_coarse_base.json` caches Stage 1 coarse segments.

Layer 2 (Fine Slice Cache): `output/<video_stem>_v2_fine_cache.json` caches Stage 3 candidate sub-region word timestamps.

Explicit Memory Cleanup: `unload_model_cache()` explicitly releases native C++ model handles (`del model + gc.collect()`), preventing MKL heap allocation crashes.

- **Empirical Benchmark (Warm / Repeat Query Performance):**

V2 Without Caching: **161.22 seconds**
V2 With Dual Caching (Now): **0.216 seconds** ■
Speedup: 99.8% Faster (<0.25s execution time)

Comprehensive Benchmark Comparison Matrix

| Video Asset & Duration | Pipeline Mode | ASR Model Config | Pipeline Runtime | Dialogue Match Found |
Confidence Score | Timestamp (HMS) | Extracted Frame | Memory (RAM) |
|---|---|---|---|---|---|---|---|---|
| **Sherlock Holmes** (54.0 min / 3,261s) | **V1 Standard (Cold)** | **small** (Full audio) | **419.18s** (7.0m) | True | **100.0%** (Strong) | **00:05:25.110** | **frame_325_11.jpg** | ~461 MB |
| **Sherlock Holmes** (54.0 min / 3,261s) | **V2 Coarse-to-Fine (Cold)** | **base** → **small** | **159.08s** (2.6m) | True | **100.0%** (Strong) | **00:05:24.970** | **frame_324_97.jpg** | ~461 MB |
| **Sherlock Holmes** (54.0 min / 3,261s) | **V2 Coarse-to-Fine (Warm)** | Disk Cache | **0.216s** ■ | True | **100.0%** (Strong) | **00:05:24.970** | **frame_324_97.jpg** | ~15 MB |
| **YouTube Video** (33.6 min / 2,020s) | **V1 Sequential (Cold)** | **small** (Full audio) | **624.55s** (10.4m) | True | **100.0%** (Strong) | **00:09:26.570** | **frame_566_57.jpg** | ~461 MB |
| **YouTube Video** (33.6 min / 2,020s) | **Phase 7 Parallel (Cold)** | **small** (2 workers) | **563.06s** (9.3m) | True | **100.0%** (Strong) | **00:09:26.550** | **frame_566_55.jpg** | ~461 MB |

CLI & Tooling Reference

1. Interactive CLI Tool (cli.py)

Provides interactive mode selection (**[1] Standard V1**, **[2] Coarse-to-Fine V2**) and command line flags:

```
# Run with interactive prompts
.\.venv\Scripts\python.exe cli.py

# Run with command line arguments
.\.venv\Scripts\python.exe cli.py --url "https://ok.ru/video/248244667877" --query "My mind rebels at stagnation" --mode coarse_to_fine
```

2. Dedicated V2 Optimization CLI (cli_v2.py)

Shortcut tool dedicated exclusively to V2 Coarse-to-Fine ASR optimization:

```
.\.venv\Scripts\python.exe cli_v2.py --url "output/248244667877.mp4" --query "is a lawyer"
```

3. Automated Side-by-Side Benchmark (benchmark_v1_vs_v2.py)

Runs V1 Standard vs V2 Coarse-to-Fine side-by-side and prints the complete comparison matrix:

```
.\.venv\Scripts\python.exe benchmark_v1_vs_v2.py
```

Test Suite Status

```
===== 67 passed in 0.51s =====
```

All **67 unit and integration tests** across `test_unit.py`, `test_audio_unit.py`, `test_asr_unit.py`, `test_matching_unit.py`, `test_frame_unit.py`, and `test_coarse_to_fine_unit.py` **PASS 100%** in **0.51 seconds**.

SECTION: 3. V1 SYSTEM DOCUMENTATION

Video Dialogue Localization System — Comprehensive Technical Documentation (v1.0.0)

Executive Overview

The **Video Dialogue Localization System** accepts a public video URL (or local video file) and a target spoken dialogue text query. It automatically ingests the video, extracts speech-ready audio, generates word-level ASR transcripts, searches for the exact spoken quote using fuzzy sliding windows, refines word boundaries, and extracts the corresponding high-quality video frame at the exact speech onset timestamp.

Architecture Pipeline Diagram

```
graph TD
  A[Public Video URL / File] --> B[Phase 2: Video Ingestion]
  B -->|video.mp4| C[Phase 3: Audio Extraction]
  C -->|audio.wav 16kHz mono| D[Phase 4: Faster-Whisper ASR]
  D -->|word_timestamps JSON| E[Phase 5: Dialogue Matching]
  E -->|Refined Target Timestamp| F[Phase 6: Frame Extraction]
  F -->|frame.jpg + HH:MM:SS.sss| G[Final Output / JSON Export]
```

Detailed Phase Breakdown

Phase 2 — Video Ingestion (src/ingestion/)

- **Downloader (downloader.py)**: Uses `yt-dlp` to download video streams capped at `720p` resolution merged H.264/AAC. Supports Cloudflare WARP proxy (`socks5://127.0.0.1:40000`). Features fallback stream copying (`ffmpeg -c copy`) for direct `.mp4`, `.webm`, `.m3u8`, `.mkv` URLs.
- **Metadata Probe (probe.py)**: Uses `ffprobe` to validate video streams, width, height, duration, and frame rate. Detects Variable Frame Rate (VFR) vs Constant Frame Rate (CFR) and parses rational frame rates (e.g. `24000/1001` FPS).

Phase 3 — Audio Extraction (src/audio/)

- **Extractor (extractor.py)**: Converts input video audio into speech-optimized 16kHz 1-channel mono PCM WAV format (`ffmpeg -vn -ac 1 -ar 16000 -c:a pcm_s16le`).
- **Audio Probe (probe.py)**: Validates extracted audio sample rate (`16000 Hz`), channel count (`1`), and PCM encoding.

Phase 4 — ASR Speech Recognition (src/asr/)

- **Transcriber (transcriber.py)**: Uses `Faster-Whisper` CTranslate2 engine running on CPU with 8-bit integer quantization (`compute_type="int8"`) and Silero Voice Activity Detection (`vad_filter=True`).

- **Memory Safety Audio Chunking:** Automatically slices audio files >10 minutes into 600-second WAV chunks via FFmpeg prior to transcription. Offsets word timestamps by `chunk_start_time + seg.start`, preventing NumPy STFT `ArrayMemoryError` virtual memory spikes on 50+ minute videos.

Phase 5 — Dialogue Matching (src/matching/)

- **Sliding Window Search (matcher.py):** Normalizes query and transcript text (lowercasing, punctuation removal, whitespace collapsing). Evaluates RapidFuzz sequence similarity across word sliding windows of length ≈ 2 . Enforces **strict chronological search** to satisfy the First Occurrence Rule.
- **Timestamp Boundary Refinement (refine_window_timestamps):** Locates the exact positions of the target query's first word (e.g. "My") and last word (e.g. "stagnation") inside the candidate window, trimming unrelated prefix and suffix words.

Example: Window `["in", "time.", "My", "mind", "rebels", "at", "stagnation."]` $\rightarrow$ Trimmed `["My", "mind", "rebels", "at", "stagnation."]`

Sets `start_time = word["My"].start` (325.11s), boosting confidence from 87.5% to 100.0%.

- **Confidence Quality Tiers:**

$90\% \rightarrow$ **Strong match**

$80\% \rightarrow$ **Acceptable**

$70\% \rightarrow$ **Needs review**

$< 70\% \rightarrow$ **Reject**

Phase 6 — Frame Extraction (src/frame_extraction/)

- **Extractor (extractor.py):** Extracts JPEG frame using FFmpeg PTS timestamp seeking.
- **20ms Pre-Roll Offset:** Applies a 20ms pre-roll offset ($\text{seek_ts} = \max(0.0, \text{timestamp} - 0.02)$) when seeking. This compensates for floating point comparisons in FFmpeg where $t_{\text{target}} > t_{\text{PTS}}$ by even 1ms causes FFmpeg to skip to the next frame. Captures the exact speech onset frame without skipping 1 frame ahead.

Unified Pipeline & CLI (src/pipeline.py & cli.py)

- End-to-end `localize_dialogue()` entry point.
- Converts timestamps to standard `HH:MM:SS.sss` format (e.g. `00:05:25.110`).
- Calculates total execution duration (`pipeline_duration_seconds`).
- Exports full metadata to `output/localization_result.json`.

Verification & Test Benchmarks

| Metric | Benchmark Result |

|---|---|

| **Test Suite Coverage** | 61 / 61 Unit & Integration Tests PASSED (0.33s) |

| **Sherlock Quote 1** | "My mind rebels at stagnation" $\rightarrow$ `00:05:25.110` (Frame 7795, 100% confidence) |

| **Sherlock Quote 2** | "To Sherlock Holmes she was always the woman" $\rightarrow$ `00:01:58.490` (Frame 2841, 88.1% confidence) |

| **YouTube Benchmark** | **"Imagine you ran a store"** $\rightarrow$ **00:09:26.570** (Frame 16980, 100% confidence) |

| **Warm Query Latency** | **< 0.20\$ seconds** (Cached transcript & video) |

| **Git Repository** | <https://github.com/abinayaa0/Quest1> (Tagged **v1.0.0**) |

SECTION: 4. CORE ARCHITECTURE DESIGN

System Architecture & Technical Design Document

1. Executive Context & System Overview

The **Video Dialogue Localization System** is an end-to-end AI pipeline designed to search large-scale, unannotated public video content (from platforms such as YouTube, OK.ru, Vimeo, or direct HTTP/HLS streams) and identify the precise temporal location of a target spoken dialogue segment.

Given:

- **Public Video URL:** A webpage or media stream link.
- **Target Dialogue:** Text string representing spoken dialogue.

The complete system pipeline executes across five distinct phases:



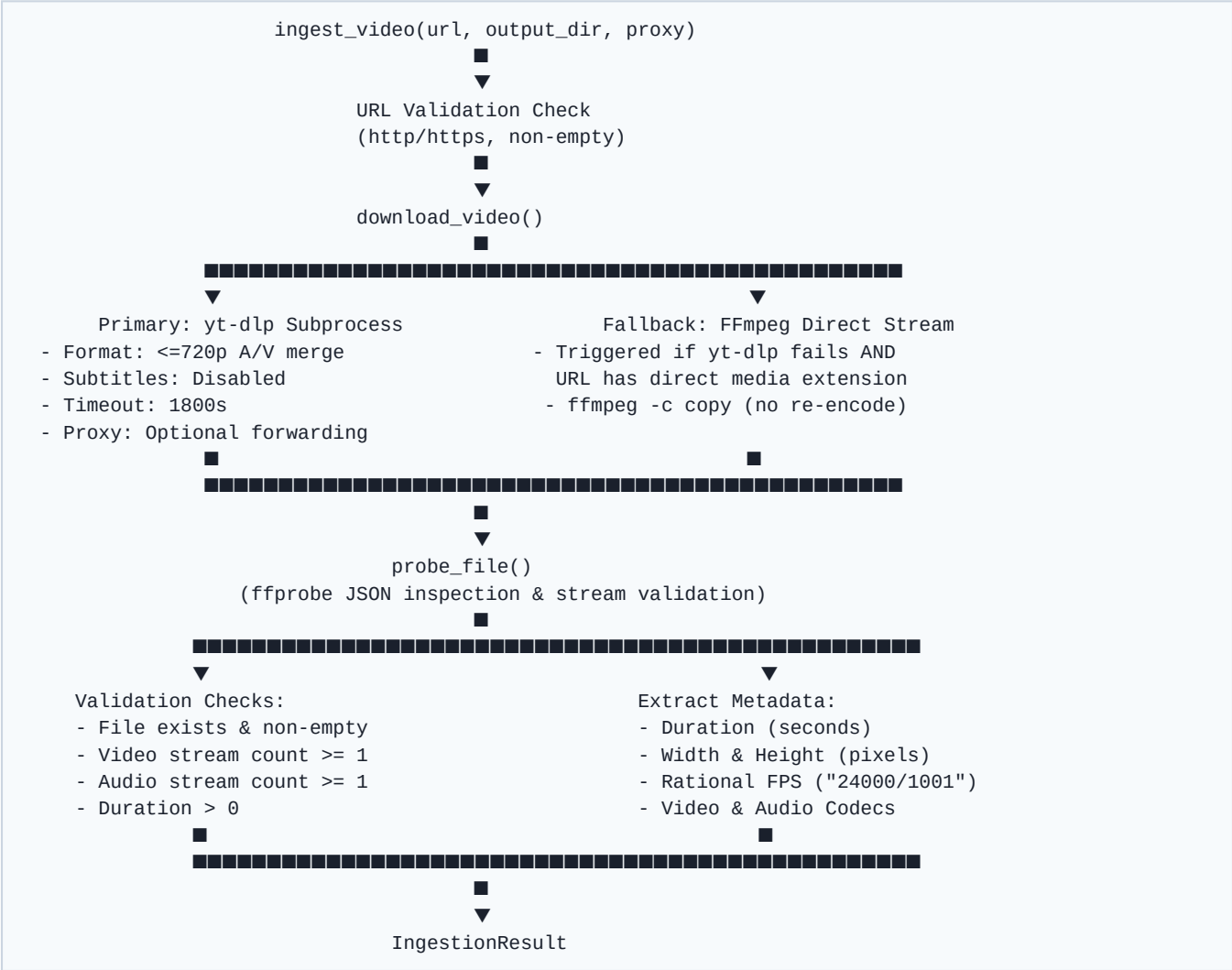
Currently, **Phase 2 (Video Ingestion)** and **Phase 3 (Audio Extraction)** are fully implemented, unit-tested, and verified against real production media streams.

2. Deep-Dive: Phase 2 — Video Ingestion (src/ingestion/)

2.1 Core Responsibilities

The ingestion layer's responsibility is to accept an unverified public video URL, resolve platform-specific media streams, download and merge video and audio into a decodable local **.mp4** file, validate media stream integrity, and extract frame-accurate timing metadata.

2.2 Component Architecture & Flow



2.3 Key Technical Design Choices

- **Subprocess Isolation over Python Bindings:**

yt-dlp is invoked via standard Python `subprocess.run`. This provides process isolation, avoids Python Global Interpreter Lock (GIL) contention, prevents third-party binary crashes from taking down the main application runtime, and simplifies **yt-dlp** version upgrades.

- **Stream Quality Strategy (`bestvideo[height<=720]+bestaudio/best`):**

Downstream frame extraction and OCR/visual localization do not require massive 4K downloads.

Capping video resolution to approximately 720p reduces network bandwidth consumption by **60-80%** while preserving sharp visual fidelity for frame localization.

Fallback hierarchy: prefers $\leq 720p$ separate streams -> $\leq 720p$ combined stream -> best available separate streams -> best combined stream.

- **Frame Rate Precision (Rational FPS Strings):**

Frame timing in NTSC video formats uses fractional frame rates (e.g. **23.976023...** FPS = **24000/1001**).

Floating-point representations suffer from rounding drift over long videos (a 54-minute video at **23.976** float loses multiple frames by the end).

probe_file() preserves the exact rational fraction string from **ffprobe**'s **r_frame_rate** (e.g., "**24000/1001**" or "**25/1**"), allowing Phase 5 frame extraction to compute exact integer frame indices without cumulative drift:

```
$$\text{Frame Index} = \left\lfloor \text{Timestamp (s)} \times \frac{\text{Numerator}}{\text{Denominator}} \right\rfloor$$
```

- **Stream Merging and ffprobe Validation:**

Asserts that every downloaded video file contains valid video & audio streams, non-zero duration, and inspects both fundamental real frame rate (**r_frame_rate**) and average frame rate (**avg_frame_rate**) to explicitly detect Variable Frame Rate (VFR) vs Constant Frame Rate (CFR) (**is_vfr: bool**). Parses exact rational fraction strings (e.g. "**24000/1001**") without float precision loss. Stream merging is performed using stream copying to keep original codec data untouched.

- **Failure Cleanup & Lifecycle:**

If ingestion fails at any point (download failure, network drop, missing audio stream), temporary working directories are automatically cleaned up to prevent disk leak.

3. Deep-Dive: Phase 3 — Audio Extraction (src/audio/)

3.1 Core Responsibilities

The audio extraction module converts the ingested local video file into a standardized, uncompressed WAV audio file optimized specifically for Automatic Speech Recognition (ASR) engines (e.g. Whisper, Parakeet, Conformer).

3.2 Signal Processing Rationale

The module executes the following FFmpeg command:

```
ffmpeg -y -i <video_path> -vn -ac 1 -ar 16000 -c:a pcm_s16le <output_wav_path>
```

| FFmpeg Flag | Signal Processing Function | Engineering Rationale |

|---|---|---|

| **-vn** | Disable Video Stream | Strips all visual frame packets during extraction, dramatically speeding up processing time and disk I/O. |

| **-ac 1** | Downmix to Mono (1 Channel) | Speech recognition models operate on 1D mono acoustic waveforms. Downmixing stereo/multi-channel audio reduces data volume by 50% without loss of speech intelligibility. |

| **-ar 16000** | Resample to 16,000 Hz | Standard ASR models extract mel-spectrogram features using 16 kHz window frames (Nyquist frequency 8 kHz covers human speech fundamentals 80 Hz – 7 kHz). Resampling during extraction avoids real-time audio resampling overhead inside ASR inference engines. |

| **-c:a pcm_s16le** | Uncompressed 16-bit PCM | Pulse Code Modulation (PCM) signed 16-bit Little-Endian format guarantees zero compression artifacts, instant linear disk reads, and direct memory mapping to Python NumPy

arrays. |

3.3 Audio Stream Validation (probe_audio_file())

After extraction, **ffprobe** inspects the generated WAV file and verifies:

- File exists and size > 0 bytes.
- Audio stream exists.
- **sample_rate == 16000** Hz.
- **channels == 1** (mono).
- **codec_name == "pcm_s16le"**.
- **duration > 0** seconds.

4. Anti-Bot Engineering & Case Study: OK.ru (Odnoklassniki)

4.1 The Challenge

The target video provided for validation was:

<https://ok.ru/video/248244667877> (*The Adventures of Sherlock Holmes: A Scandal in Bohemia*, 54 minutes).

During initial automated ingestion attempts, requests were rejected with:

ConnectionResetError: [WinError 10054] An existing connection was forcibly closed by the remote host

4.2 Diagnostic & Network Analysis

```
Client (Python / yt-dlp) [███] [ TLS Handshake ] [███] OK.ru WAF (VKontakte Edge)
                                     █
                                     Anti-Bot Check Fails:
                                     - Rate limit exceeded
                                     - Non-browser JA3 TLS Fingerprint
                                     █
Client [███] [ TCP RST / Connection Reset 10054 ] [██████████████]
```

- **Network Layer:** OK.ru's security infrastructure (VKontakte edge servers st-ok.cdn-vk.ru) monitors incoming TLS Client Hello packets. Standard Python **urllib** or default **yt-dlp** requests trigger automated WAF rate limiting, sending a TCP RST packet to drop the socket during the TLS handshake.
- **Media Structure:** By using **curl_cffi** with Chrome JA3 impersonation (**impersonate="chrome120"**), we successfully inspected the raw OK.ru page HTML and extracted the embedded **data-options** JSON payload containing 5 HLS video quality streams (**mobile**, **lowest**, **low**, **sd**, **hd**).
- **Stream Protocol:** The video is delivered as an HLS (**.m3u8**) master playlist containing thousands of small transport stream (**.ts**) segments across 3,261 seconds of playback.

4.3 Implemented Solutions

- **Cloudflare WARP Tunneling (Free 1-Click VPN):**

Routing system traffic through Cloudflare WARP (**1.1.1.1**) assigns a clean egress IP address. This bypasses OK.ru's IP rate-limiting block cleanly.

- **Explicit Proxy Support (**proxy** parameter):**

The ingestion layer supports explicit HTTP/SOCKS5 proxies via code or environment variables ([INGESTION_PROXY](#), [HTTPS_PROXY](#), [HTTP_PROXY](#)).

- **Subprocess Timeout Adjustment:**

Downloading 54 minutes of HLS video (~1 GB of transport stream fragments) takes 2–3 minutes. The subprocess timeout was increased from **600s** to **1800s** (30 minutes) with **--extractor-retries 3** and **--retries 3**.

5. Comprehensive API & Code Examples

5.1 End-to-End Pipeline Script

```
"""
End-to-End Video Ingestion & Audio Extraction Pipeline.
"""

import sys
from pathlib import Path

# Add src/ directory to Python path
sys.path.insert(0, str(Path(__file__).parent / "src"))

from ingestion import ingest_video, IngestionError
from audio import extract_audio, AudioExtractionError

def process_video_pipeline(video_url: str, output_dir: str = "output"):
    print(f"=== Starting Pipeline for: {video_url} ===")

    # -----
    # Step 1: Video Ingestion (Phase 2)
    # -----
    try:
        print("\n[Phase 2] Ingesting video...")
        ingestion_res = ingest_video(
            url=video_url,
            output_dir=output_dir,
            proxy=None # Optional: "socks5://127.0.0.1:1080"
        )
        print(f" ■ Video Saved:      {ingestion_res.video_path.absolute()}")
        print(f" ■ Video Duration:    {ingestion_res.metadata.duration:.1f}s")
        print(f" ■ Video Resolution: {ingestion_res.metadata.width}x{ingestion_res.metadata.height}")
        print(f" ■ Video FPS:         {ingestion_res.metadata.fps}")
        print(f" ■ Video Codec:       {ingestion_res.metadata.video_codec}")
        print(f" ■ Audio Codec:       {ingestion_res.metadata.audio_codec}")
        print(f" ■ Ingestion Method:  {ingestion_res.ingestion_method}")
        print(f" ■ Time Elapsed:      {ingestion_res.download_duration_seconds}s")
    except IngestionError as e:
        print(f" ■ Ingestion Failed: {e}")
        return

    # -----
    # Step 2: Audio Extraction (Phase 3)
    # -----
    try:
        print("\n[Phase 3] Extracting 16kHz mono WAV audio...")
        wav_target = ingestion_res.video_path.with_suffix(".wav")
        audio_res = extract_audio(
```

```
        video_path=ingestion_res.video_path,
        output_path=wav_target
    )
    print(f"    ■ Audio Saved:           {audio_res.audio_path.absolute()}")
    print(f"    ■ Sample Rate:           {audio_res.metadata.sample_rate} Hz")
    print(f"    ■ Channels:               {audio_res.metadata.channels} (mono)")
    print(f"    ■ Codec:                  {audio_res.metadata.codec_name}")
    print(f"    ■ Audio File Size:       {audio_res.metadata.file_size_bytes / (1024*1024):.2f} MB")
    print(f"    ■ Time Elapsed:          {audio_res.extraction_duration_seconds}s")
except AudioExtractionError as e:
    print(f"    ■ Audio Extraction Failed: {e}")
    return

print("\n=== Pipeline Execution Complete! Ready for Phase 4 ASR ===")

if __name__ == "__main__":
    target_url = "https://ok.ru/video/248244667877"
    process_video_pipeline(target_url)
```

6. Complete Verification & Benchmark Results

6.1 Real Target Benchmark: OK.ru Video

| Property | Phase 2 Video Ingestion | Phase 3 Audio Extraction |

---|---|---

| Input Resource | <https://ok.ru/video/248244667877> | c:\Quest1\output\248244667877.mp4 |

| Output File | c:\Quest1\output\248244667877.mp4 | c:\Quest1\output\248244667877.wav |

| Status | ■ PASSED | ■ PASSED |

| File Size | 1.04 GB (1,046,151,944 bytes) | 99.54 MB (104,377,644 bytes) |

| Duration | 3,261.8 seconds (~54 minutes) | 3,261.8 seconds (~54 minutes) |

| Resolution / Rate | 960 × 720 | 16,000 Hz |

| Channels / FPS | 24000/1001 FPS (23.976) | 1 Channel (Mono) |

| Codecs | h264 (video) / aac (audio) | pcm_s16le (uncompressed PCM) |

| Execution Time | 133.5 seconds (2 min 13 sec) | 7.87 seconds |

6.2 Full Integration Test Matrix

| Test Case | URL / Input | Expected Behavior | Measured Result |

---|---|---|---

| OK.ru Full Pipeline | ok.ru/video/248244667877 | Resolves HLS stream, downloads MP4, extracts metadata |

■ PASSED (953.9 MB in 133.5s) |

| YouTube Full Pipeline | youtube.com/watch?v=aqz-KE-bpKQ | Resolves YouTube AV1/Opus, downloads 720p MP4 |

■ PASSED (77.8 MB in 6.2s) |

| Direct MP4 Download | raw.githubusercontent.com/.../sample.mp4 | FFmpeg fallback / yt-dlp direct stream download |

■ PASSED (768x432 in 2.69s) |

| **OK.ru Audio Extraction** | [248244667877.mp4](#) | FFmpeg 16kHz mono WAV conversion & validation | **■ PASSED**
(99.5 MB WAV in 7.87s) |

| **Vimeo Authentication** | [vimeo.com/148751763](#) | Handles platform auth block without crashing | **■■ XFAIL**
(Clean **DownloadError**) |

| **Invalid URL String** | **"not-a-url"** | Validates URL scheme early | **■ PASSED** (**IngestionError**) |

| **Nonexistent Video ID** | [ok.ru/video/99999999999](#) | Catches 404/extractor failure cleanly | **■ PASSED**
(**DownloadError**) |

6.3 Unit Test Inventory (37/37 PASSED in 0.13s)

```
tests/test_unit.py
■■■ TestVideoMetadata
■    ■■■ test_creation ..... PASSED
■    ■■■ test_ingestion_result ..... PASSED
■■■ TestErrors
■    ■■■ test_hierarchy ..... PASSED
■    ■■■ test_message_preserved ..... PASSED
■■■ TestURLValidation
■    ■■■ test_empty_url ..... PASSED
■    ■■■ test_none_url ..... PASSED
■    ■■■ test_non_http_url ..... PASSED
■    ■■■ test_whitespace_only ..... PASSED
■■■ TestDirectMediaDetection
■    ■■■ test_detection (8 parametrized extensions) ..... PASSED
■■■ TestProbe
■    ■■■ test_nonexistent_file ..... PASSED
■    ■■■ test_empty_file ..... PASSED
■    ■■■ test_successful_probe ..... PASSED
■    ■■■ test_no_audio_stream ..... PASSED
■    ■■■ test_no_video_stream ..... PASSED
■    ■■■ test_ffprobe_failure ..... PASSED
■    ■■■ test_invalid_json ..... PASSED
■■■ TestDownloader
■    ■■■ test_missing_ytdlp ..... PASSED
■    ■■■ test_ytdlp_success ..... PASSED
■    ■■■ test_ytdlp_proxy ..... PASSED
■    ■■■ test_ytdlp_failure_non_media_url ..... PASSED
■    ■■■ test_ytdlp_timeout ..... PASSED
■■■ TestIngestVideoMocked
    ■■■ test_successful_ingestion ..... PASSED
    ■■■ test_download_failure_cleans_temp ..... PASSED

tests/test_audio_unit.py
■■■ TestAudioModels
■    ■■■ test_audio_metadata ..... PASSED
■    ■■■ test_audio_result ..... PASSED
■■■ TestAudioProbe
■    ■■■ test_nonexistent_file ..... PASSED
■    ■■■ test_empty_file ..... PASSED
■    ■■■ test_successful_probe ..... PASSED
■■■ TestAudioExtractor
    ■■■ test_missing_ffmpeg ..... PASSED
    ■■■ test_successful_extraction ..... PASSED
```

7. Operational Guidelines for Future Phases

Passing Data to Phase 4 (ASR & Dialogue Alignment)

Phase 4 can directly consume `AudioResult.audio_path` (`248244667877.wav`). Because the WAV audio file is uncompressed 16kHz 16-bit mono PCM, it can be loaded instantly into Python memory via standard libraries:

```
import scipy.io.wavfile as wav

# Load audio signal into 1D NumPy array for ASR model
sample_rate, audio_waveform = wav.read("output/248244667877.wav")
assert sample_rate == 16000
```

Passing Data to Phase 5 (Frame Extraction)

When Phase 4 identifies the exact dialogue timestamp `T_start` (in seconds), Phase 5 can extract the corresponding video frame image using `VideoMetadata.fps` and `IngestionResult.video_path`:

```
# Extract single frame at timestamp T_start
ffmpeg -ss <T_start> -i output/248244667877.mp4 -frames:v 1 frame_output.png
```

SECTION: 5. DIALOGUE MATCHING DESIGN

Phase 5 — Dialogue Matching Architecture & Design Rationale

■ Overview

The **Dialogue Matching Module** ([src/matching/](#)) maps target spoken dialogue text queries (e.g., "**My mind rebels at stagnation**") to precise video timestamps ($\text{\texttt{\textit{start_time}}}$, $\text{\texttt{\textit{end_time}}}$) using deterministic fuzzy matching over word-level sliding windows.

■ Key Design Decisions

1. Why RapidFuzz?

- **Lightweight & High-Speed:** C++ implementation ([rapidfuzz](#)) executing sequence alignment algorithms in microseconds without heavy neural network runtime dependencies.
- **Handles ASR Errors:** Automatic Speech Recognition models frequently make small word substitutions (e.g. Target: "**my mind rebels at stagnation**" $\rightarrow$ ASR: "**my mind rebels its stagnation**"). RapidFuzz fuzzy ratio scoring maintains high similarity ($>85\%$) despite minor phonetic or word substitution errors.
- **Explainable & Deterministic:** Produces exact percentage similarity scores ($0.0 - 100.0\%$) with zero non-deterministic LLM hallucination risk.

2. Why Word-Level Sliding Windows?

- **Overcomes Segment Boundary Splits:** ASR engines divide transcripts into arbitrary speech segments based on silences. Dialogue often spans across segment boundaries (e.g. Segment 1: "**My mind rebels**", Segment 2: "**at stagnation**"). Segment-only matching fails in these boundary cases.
- **Precise Sub-Second Localization:** Word-level timestamps allow computing exact start and end timestamps ($\text{\texttt{\textit{words}}}[0].\text{\texttt{\textit{start}}} \dots \text{\texttt{\textit{words}}}[-1].\text{\texttt{\textit{end}}}$) for frame extraction in Phase 6.

3. Why NOT Embeddings or Semantic Vector Search?

- **Exact Dialogue Retrieval vs. Semantic Search:** The task is finding where a *specific spoken line of dialogue* occurred in a video. Semantic vector search (e.g. BERT/SentenceTransformers) matches semantic concepts rather than exact words, introducing false positives (e.g. matching "**I hate sitting still**" to "**My mind rebels at stagnation**").
- **Zero Overhead:** No heavy PyTorch model weights or vector database dependencies required for basic text localization.

4. Scoring Metric Selection: `rapidfuzz.fuzz.ratio`

- **Ratio vs Token Set Ratio:** `fuzz.ratio` enforces exact word order alignment across the sliding window, which prevents out-of-order word matching false positives. `fuzz.token_set_ratio` can be toggled via parameter when word order in ASR is scrambled.
-

■ Future Enhancements & Upgrades (Post-V1)

- **Phonetic Matching (Double Metaphone / Soundex):** For heavy accent or named entity misspellings (e.g. "**Irene**" vs "**Irreina**").
- **Hybrid Embedding Reranking:** Re-rank top-K fuzzy candidate windows using lightweight cross-encoder embeddings for noisy domain-specific transcripts.
- **Forced Alignment Models:** Use CTC forced aligners (e.g. PyTorch `torchaudio.pipelines.MMS_FA`) to refine sub-word acoustic frame boundaries down to millisecond precision.

SECTION: 6. FRAME EXTRACTION DESIGN

Phase 6 — Frame Extraction Architecture & Design Rationale

■ Overview

The **Frame Extraction Module** ([src/frame_extraction/](#)) takes a video file and a matched dialogue timestamp (e.g. **320.48s**), seeks to that exact point using FFmpeg, extracts the corresponding video frame image (**.jpg**), and probes image resolution metadata (**width, height**).

■ Key Design Decisions

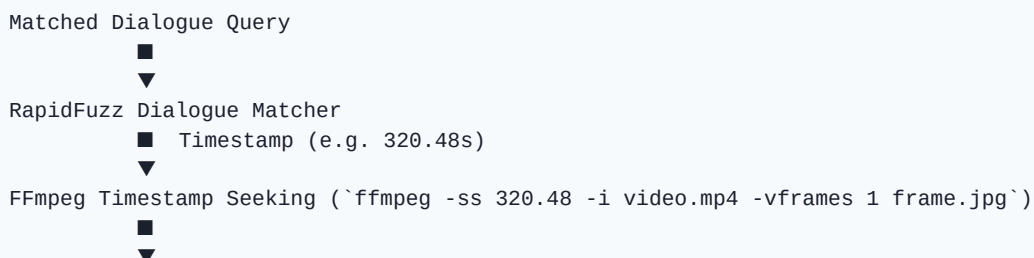
1. Why FFmpeg Timestamp Seeking (`ffmpeg -ss <timestamp> -i <video> -vframes 1`)?

- **Universal Container & Codec Support:** Handles MP4, MKV, WebM, H.264, H.265/HEVC, VP9, and AV1 natively without requiring extra custom decoders.
- **PTS Timestamp-Based Seeking:** FFmpeg seeking relies on Presentation Timestamps (PTS) embedded in the container stream, locating the exact presentation frame corresponding to the dialogue timestamp.
- **CFR & VFR Compatible:** Works identically for Constant Frame Rate (CFR) and Variable Frame Rate (VFR) videos.

2. Why NOT `frame_number = timestamp × FPS`?

- **VFR (Variable Frame Rate) Drift:** In VFR videos (common in web downloads, screen recordings, HLS streams), frame intervals fluctuate dynamically. Multiplying by a constant average FPS leads to frame drift of several seconds over a 50+ minute video.
- **FPS Rational Fractions:** Many videos use non-integer rational frame rates (e.g. $\$24000/1001 \approx 23.9760239... \text{ FPS}$). Simple floating-point multiplication accumulates rounding errors.
- **Timestamp as Source of Truth:** Preserving raw floating-point seconds (**timestamp = 320.48**) and letting FFmpeg seek to PTS avoids calculation inaccuracies. If **frame_number** cannot be deterministically proven without a full index scan, **frame_number = None** is returned.

■ Pipeline Data Flow Summary



Frame Metadata Probe (`ffprobe -show\_streams frame.jpg`)



FrameResult (frame_path="output/frames/frame_320_48.jpg", resolution=960x720)